

Informatika 3

Prednáška 9: Štandardná knižnica jazyka C++

Michal Mrena



ŽILINSKÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

Knižnice

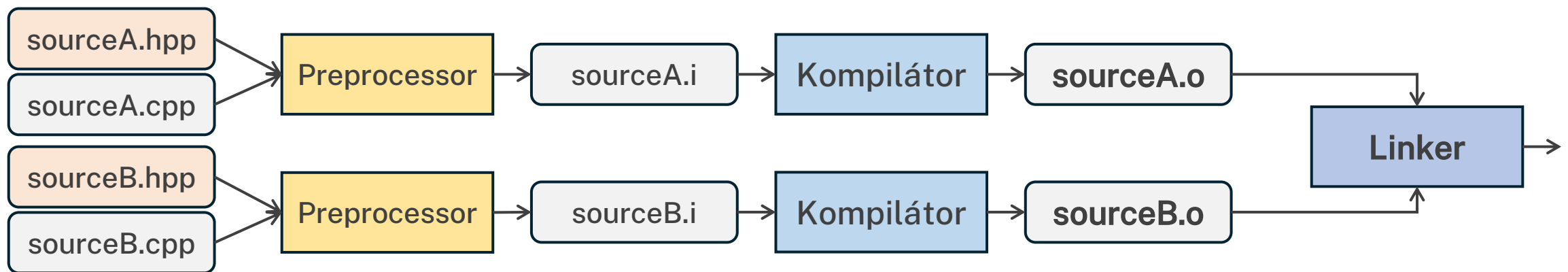


Knižnice

- Takmer všetky programy využívajú kód (funkcie, triedy), ktoré napísal iný programátor
- Bez nich by bolo vytvorenie sofistikovanejšieho programu veľmi náročné
- Takýto kód nazývame (softvérová) knižnica
- V závislosti od platformy, jazyka, nástrojov, charakteru a *spôsobu šírenia* môže mať rôzne formy:
 - Binárny súbor
 - Archív (binárne súbory, textové súbory, iné zdroje)
 - Zdrojový kód

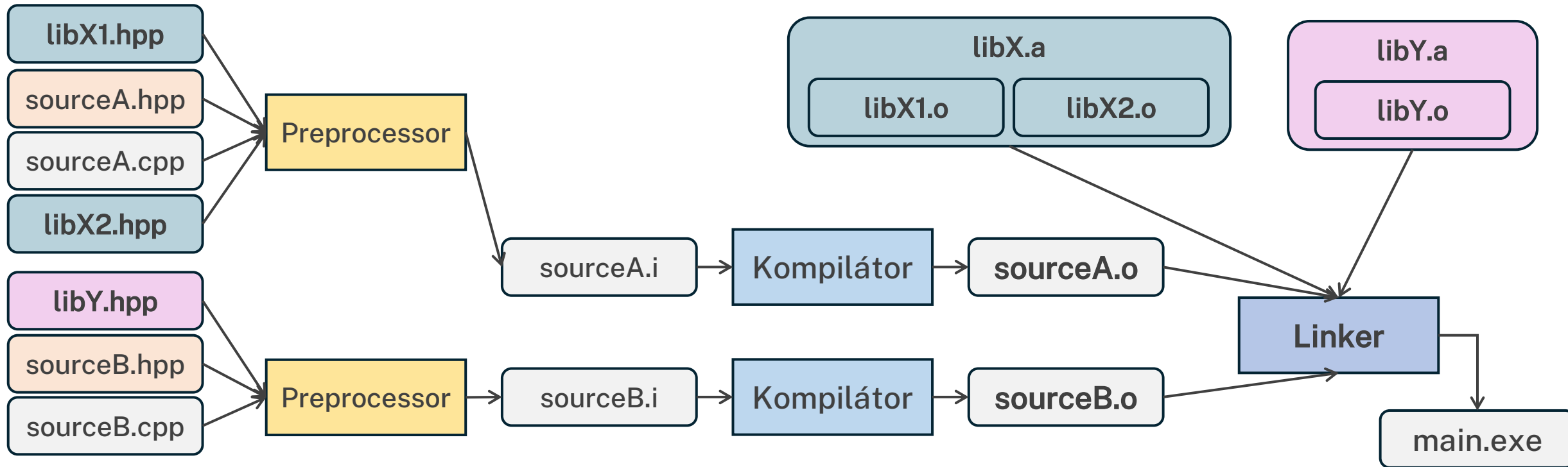
Knižnice v jazyku C++

- V jazyku C++ rozlišujeme niekoľko typov knižníc
- Rozdiel medzi nimi je daný procesom prekladu – veľmi dôležitý je krok linkovania
- V praxi sa používajú všetky
- Vo väčších projektoch je práca s knižnicami (závislosťami) často náročná a problematická



Statické knižnice

- Statické (alebo staticky linkované knižnice) knižnice pomerne intuitívne vyplývajú z procesu prekladu



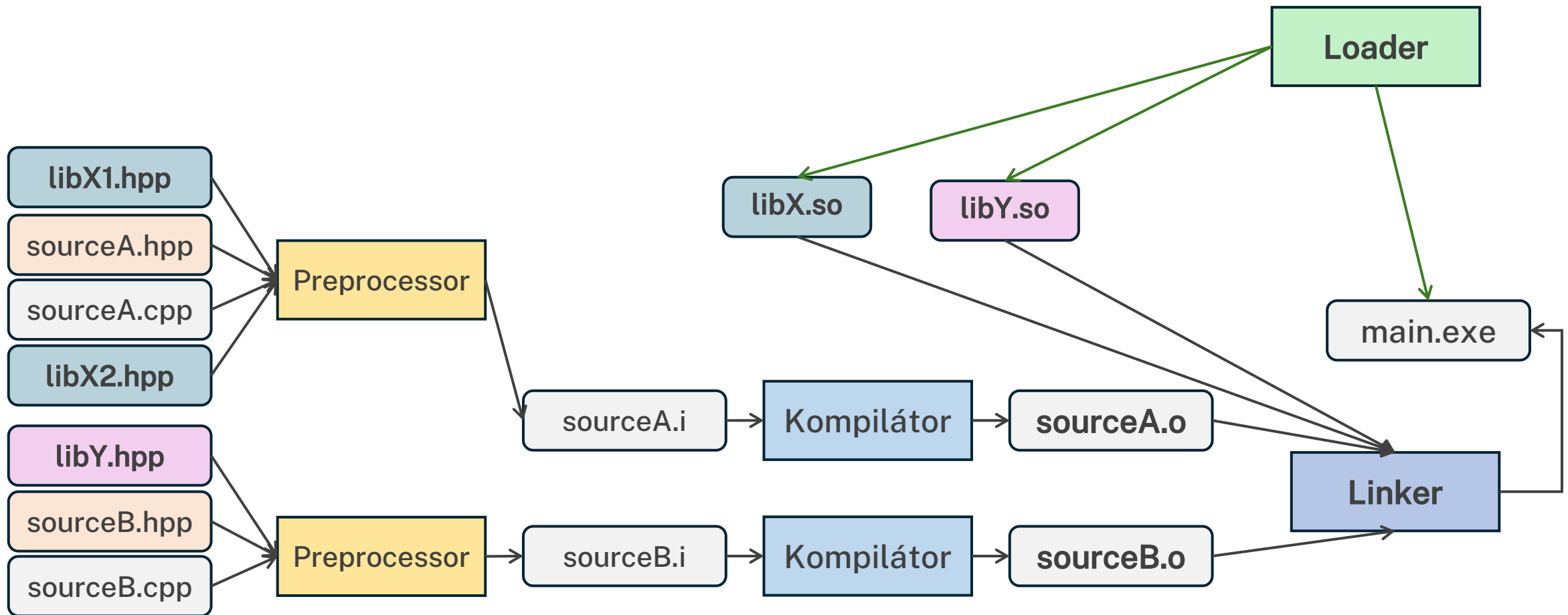
Statické knižnice

- Statická knižnica je **archív** (.a – Linux, .lib – Windows) obsahujúci objektové súbory (.o – Linux, .obj – Windows) obsahujúce **definície**
- Na jej použitie je potrebné mať k dispozícii hlavičkové súbory (.hpp) obsahujúce **deklarácie**
- **Používateľ knižnice** vloží do svojho kódu hlavičkový súbor s deklaráciami
- Priradenie definícií zabezpečí **linker** vo fáze linkovania

Statické knižnice – vlastnosti

- Všetky potrebné funkcie sú súčasťou výstupu linkera (spustiteľného súboru alebo knižnice)
- Distribúcia knižnice je jednoduchšia
- Menej problémov s nekompatibilnými verziami a chýbajúcimi funkciami
- Spustiteľný súbor je väčší (na druhej strane obsahuje iba funkcie, ktoré sú skutočne použité)
- Zmena v knižnici vyústi v opätovnú kompiláciu spustiteľného súboru

Dynamické knižnice



Dynamické knižnice

- Dynamická (zdieľaná) knižnica (`.so` – Linux, `.dll` – Windows, `.dylib` – macOS) je vo formáte podobnom spustiteľnému súboru
- Na jej použitie je potrebné mať k dispozícii hlavičkové súbory (`.hpp`) obsahujúce deklarácie
- Používateľ knižnice vloží do svojho kódu hlavičkový súbor s deklaráciami
- Linker do spustiteľného súboru vloží volanie funkcie a názov dynamicky linkovanej knižnice
- Po spustení programu **loader** (dynamický linker – súčasť OS) zabezpečí načítanie knižnice do pamäťového priestoru procesu

Dynamické knižnice – vlastnosti

- Zmenšujú duplicitu kódu – niekoľko rôznych procesov môže používať tú istú dynamickú knižnicu – obzvlášť výhodné pri často používaných knižniciach
- Zmena implementácie knižnice (pri zachovaní ABI compatibility) sa nedotkne spustiteľného súboru, ktorý ju používa*
- Použité knižnice musia byť nainštalované – **v kompatibilnej verzii** – v systéme na ktorom sa program spúšťa
- Načítavanie funkcií počas behu môže mať vplyv na výkon
- Často sa používajú na implementáciu pluginov

O zaujímavom využití tejto vlastnosti si môžete viac prečítať [tu](#).*

Distribúcia knižníc

- Ďalším spôsobom rozdelenia knižníc je spôsob ich distribúcie
- **Binárne** – v systéme sú nainštalované binárne súbory (.a, .so, ...) a hlavičkové súbory
 - Kompilátor nájde hlavičky v predefinovaných umiestneniach (alebo umiestneniach dodefinovaných build systémom)
 - Linker nájde binárne súbory v preddefinovaných alebo dodefinovaných umiestneniach
- Tento spôsob je typický pre knižnice, ktoré sú inštalované ako balíček alebo inštalátorom

Distribúcia knižníc

- V jazyku C++ je pomerne častá aj distribúcia knižnice vo forme zdrojového kódu – typicky pri projektoch s otvoreným zdrojovým kódom
([nlohmann::json](#), [csv-parser](#))
- Používateľ takejto knižnice ju preloží pri preklade svojho kódu*
- Výhodou je lepšia možnosť optimalizácie pre cieľovú platformu
- Nevýhodou sú potenciálne problémy pri preklade a dlhší čas prekladu

Tento prístup je rozšírený v linuxovej distribúcii [Gentoo](#). *

Distribúcia knižníc

- Distribúcia zdrojového kódu je jedinou možnosťou* ak sa knižnica skladá iba z hlavičkových súborov
- Takáto situácia je veľmi častá pri knižniciach, ktoré vo veľkom využívajú šablóny
- Takéto knižnice nazývame header-only knižnice
- Ich integrácia je veľmi jednoduchá – na druhej strane môžu mať negatívny vplyv na čas prekladu
- Tieto knižnice často ponúkajú aj tzv. single-header variantu, príklad môžeme vidieť tu a tu

(od C++20) Moduly ponúkajú alternatívny a modernejší prístup.*

Správa závislostí

- Niektoré programovacie jazyky majú vo svojom ekosystéme jednoduché nástroje na automatickú inštaláciu závislostí (knižníc)
- V jazyku C++ ich existuje niekoľko:
 - [vcpkg](#)
 - [conan](#)
- Často však využívame systémového správcu balíčkov ([pacman](#), [winget](#), [apt](#), [dnf](#), [xbps](#), ...)
- Alebo [build systém](#) v kombinácii so [submodulmi](#) nástroja git

```
ansi2html==1.9.2
beautifulsoup4==4.13.5
gitdb==4.0.12
GitPython==3.1.45
ldap3==2.9.1
lxml==6.0.1
numpy==2.3.2
pandas==2.3.1
pyasn1==0.6.1
python-dateutil==2.9.0.post0
pytz==2025.2
six==1.17.0
smmap==5.0.2
soupsieve==2.7
typing_extensions==4.14.1
tzdata==2025.2
```

Ukážka súboru requirements.txt
používaným v jazyku Python.

Štandardná knižnica



Štandardná knižnica jazyka C++

- Obsah knižnice je určený štandardom – určuje čo má knižnica obsahovať (rozhranie), nie však ako presne majú byť veci implementované*
- Aj z tohto dôvodu existuje niekoľko implementácií – zvyčajne sú viazané na konkrétny kompilátor alebo platformu
- Historicky je základom štandardnej knižnice knižnica STL (1993) (z angl. Standard Template Library), ktorej autormi sú Alexander Stepanov a David Musser

Pri niektorých algoritmoch však vyžaduje určitú výpočtovú zložitosť.*

Hlavné implementácie

Názov	Skratka	Platforma	Kompilátor
GNU C++ Standard Library	libstdc++	Windows, Linux, macOS	g++, clang++
LLVM C++ Standard Library	libc++	Windows, Linux, macOS	clang++
Microsoft C++ Standard Library	MSVC STL	Windows	MSVC
NVIDIA C++ Standard Library	libcudacxx	Windows, Linux, macOS	nvcc , g++, clang++, MSVC
Electronic Arts Standard Template Library	EASTL	Windows, Linux, macOS	g++, clang++, MSVC

Podpora novších štandardov sa medzi knižnicami môže značne líšiť.

Linkovanie

- Kompilátor zabezpečí linkovanie štandardnej knižnice automaticky
- Okrem nej linkuje aj niekoľko ďalších platformovo špecifických knižníc
- Linkovanie ďalších knižníc musíme vyžiadať explicitne
- Zvyčajne túto úlohu zabezpečuje build systém
(target_link_libraries)

```
> dumpbin.exe /dependents a.exe
```

Dump of file a.exe

File Type: EXECUTABLE IMAGE

Image has the following dependencies:

KERNEL32.dll

api-ms-win-crt-environment-l1-1-0.dll
api-ms-win-crt-heap-l1-1-0.dll
api-ms-win-crt-locale-l1-1-0.dll
api-ms-win-crt-math-l1-1-0.dll
api-ms-win-crt-private-l1-1-0.dll
api-ms-win-crt-runtime-l1-1-0.dll
api-ms-win-crt-stdio-l1-1-0.dll
api-ms-win-crt-string-l1-1-0.dll

libstdc++-6.dll

Language

- Preprocessor – Comments
- ASCII chart
- Basic concepts
 - Keywords
 - Names (lookup)
 - Types (fundamental types)
 - The main function
 - Modules (C++20)
 - Contracts (C++26)
- Expressions
 - Value categories
 - Evaluation order
 - Operators (precedence)
 - Conversions – Literals
 - Constant expressions
- Statements
 - if – switch
 - for – range-for (C++11)
 - while – do-while
- Declarations – Initialization
- Functions – Overloading
- Coroutines (C++20)
- Classes (unions)
- Templates – Exceptions
- Freestanding implementations

Standard library (headers)

Named requirements

Language support library

- Program utilities
 - Signals – Non-local jumps
- Basic memory management
- Variadic functions
- source_location (C++20)
- Comparison utilities (C++20)
- Type support – type_info
- numeric_limits – exception
- initializer_list (C++11)
- Coroutine support (C++20)
- Contract support (C++26)

Concepts library (C++20)

Diagnostics library

- Assertions – System error (C++11)
- Exception types – Error numbers
- basic_stacktrace (C++23)
- Debugging support (C++26)

Memory management library

- Allocators – Smart pointers
- Memory resources (C++17)

Metaprogramming library (C++11)

- Type traits – ratio
- integer_sequence (C++14)

General utilities library

- Function objects – hash (C++11)
- Swap – Type operations (C++11)
- Integer comparison (C++20)
- pair – tuple (C++11)
- optional (C++17)
- expected (C++23)
- variant (C++17) – any (C++17)
- bitset – Bit manipulation (C++20)

Containers library

- vector – deque – array (C++11)
- list – forward_list (C++11)
- inplace_vector (C++26)
- hive (C++26)
- map – multimap – set – multiset
- unordered_map (C++11)
- unordered_multimap (C++11)
- unordered_set (C++11)
- unordered_multiset (C++11)
- Container adaptors
- span (C++20) – mdspan (C++23)

Iterators library

Ranges library (C++20)

- Range factories – Range adaptors
- generator (C++23)

Algorithms library

- Numeric algorithms
- Execution policies (C++17)
- Constrained algorithms (C++20)

Strings library

- basic_string – char_traits
- basic_string_view (C++17)

Text processing library

- Primitive numeric conversions (C++17)
- Formatting (C++20) – Localization
- text_encoding (C++26)
- Regular expressions (C++11)
 - basic_regex – Algorithms
 - Default regular expression grammar
- Null-terminated sequence utilities:
 - byte – multibyte – wide

Numerics library

- Common math functions
- Mathematical special functions (C++17)
- Mathematical constants (C++20)
- Basic linear algebra algorithms (C++26)
- Data-parallel types (SIMD) (C++26)
- Pseudo-random number generation
- Floating-point environment (C++11)
- complex – valarray

Date and time library

- Calendar (C++20) – Time zone (C++20)

Input/output library

- Print functions (C++23)
- Stream-based I/O – I/O manipulators
- basic_istream – basic_ostream
- Synchronized output (C++20)
- File systems (C++17)

Concurrency support library (C++11)

- thread – jthread (C++20)
- atomic – atomic_flag
- atomic_ref (C++20) – memory_order
- Mutual exclusion – Condition variables
- Futures – Semaphores (C++20)
- latch (C++20) – barrier (C++20)
- Safe Reclamation (C++26)

Execution support library (C++26)

Feature test macros (C++20)

- Language – Standard library – Headers

Kontajnery



Kontajnery

- Kontajnery (údajové štruktúry) sú základom každého programu
- Ich úlohou je uchovávať a sprístupňovať *objekty* (fundamentálne typy, inštancie tried)
- Štandardná knižnica poskytuje niekoľko základných kontajnerov, ktoré pokrývajú väčšinu prípadov použitia
- Všetky kontajnery sú šablóny tried

Containers library

- vector - deque - array (C++11)
- list - forward_list (C++11)
- inplace_vector (C++26)
- hive (C++26)
- map - multimap - set - multiset
- unordered_map (C++11)
- unordered_multimap (C++11)
- unordered_set (C++11)
- unordered_multiset (C++11)
- Container adaptors
- span (C++20) - mdspan (C++23)

Vektor

- Vektor je implementáciou abstraktného údajového typu zoznam*
- Údaje uchováva v súvislej pamäti (v poli), ktorého veľkosť prispôsobuje počtu prvkov
- Poskytuje **efektívny** prístup k prvkom podľa ich indexu
- Podporuje **efektívne** vkladanie[^] a mazanie prvkov na konci vektora

```
int main() {  
    std::vector<int> xs;  
    xs.push_back(10);  
    xs.push_back(20);  
    xs.push_back(30);  
    std::cout << xs.size() << "\n";  
    std::cout << xs[0] << "\n";  
    std::cout << xs.at(1) << "\n";  
    std::cout << xs.back() << "\n";  
    xs.clear();  
}
```

alebo ako kontajner `ArrayList`, ktorý poznáte z jazyka Java.

*

Až na jednu situáciu, ktorú ukážeme na ďalšej snímke. [^]

Vektor – vkladanie prvkov

- Vkladanie prvkov metódou `push_back` inicializuje prvky ich kopírovacím alebo move konštruktorom
- Metóda `emplace_back` inicializuje prvok priamo v kontajneri jeho (parametrickým konštruktorom)
- Pri *objektových* typoch je výhodnejšie použitie `emplace`

```
class Player {  
public:  
    explicit Player(int id);  
    Player(const Player &other);  
    Player(Player &&other) noexcept;  
};
```

```
int main() {  
    std::vector<Player> xs;  
    Player p1(10);  
    Player p2(30);  
    xs.push_back(p1);  
    xs.push_back(Player(20));  
    xs.push_back(std::move(p2));  
    xs.emplace_back(40);  
}
```

`std::vector<T,Allocator>::push_back`

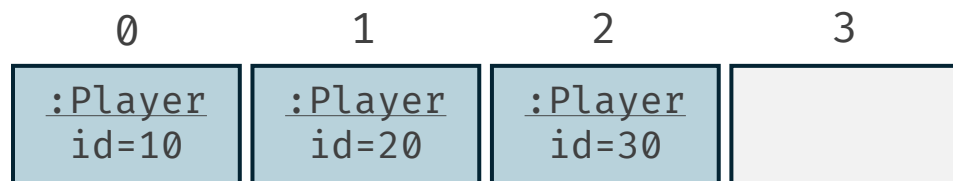
<code>void push_back(const T& value);</code>	(1)	(constexpr since C++20)
<code>void push_back(T&& value);</code>	(2)	(since C++11) (constexpr since C++20)

`std::vector<T,Allocator>::emplace_back`

<code>template< class... Args ></code>	
<code>void emplace_back(Args&&... args);</code>	(until C++17)
<code>template< class... Args ></code>	
<code>reference emplace_back(Args&&... args);</code>	(since C++17) (constexpr since C++20)

Vektor – kapacita

- Vektor na pozadí uchováva pole s istou rezervovanou kapacitou
- Ak sa pole naplní, alokuje nové (väčšie) pole a prvky *presunie* do nového poľa
 - Pri *objektových* typoch vytvorí nové inštancie a inicializuje ich move konštruktorom z pôvodných inštancií



Pole typu `Player[]` o veľkosti 4. Aktuálne sa využívajú 3 prvky, posledný je k dispozícii.

```
int main() {  
    std::vector<Player> xs;  
    Player p1(10);  
    Player p2(30);  
    xs.push_back(p1);  
    xs.push_back(Player(20));  
    xs.push_back(std::move(p2));  
}
```


Vektor – kapacita

- Vektor na pozadí uchováva pole s istou rezervovanou kapacitou
- Ak sa pole naplní, alokuje nové (väčšie) pole a prvky *presunie* do nového poľa
 - Pri *objektových* typoch vytvorí nové inštancie a inicializuje ich move konštruktorom z pôvodných inštancií

0	1	2	3
<code>:Player</code> <code>id=10</code>	<code>:Player</code> <code>id=20</code>	<code>:Player</code> <code>id=30</code>	<code>:Player</code> <code>id=40</code>

Pole typu `Player[]` o veľkosti 4. Aktuálne sa využívajú 4 prvky, pole je plne využité.

```
int main() {  
    std::vector<Player> xs;  
    Player p1(10);  
    Player p2(30);  
    xs.push_back(p1);  
    xs.push_back(Player(20));  
    xs.push_back(std::move(p2));  
    xs.emplace_back(40);  
}
```

Vektor – kapacita

- Pri (pred) vložení(m) ďalšieho prvku musí prísť k alokovaniu nového poľa – k expanzii

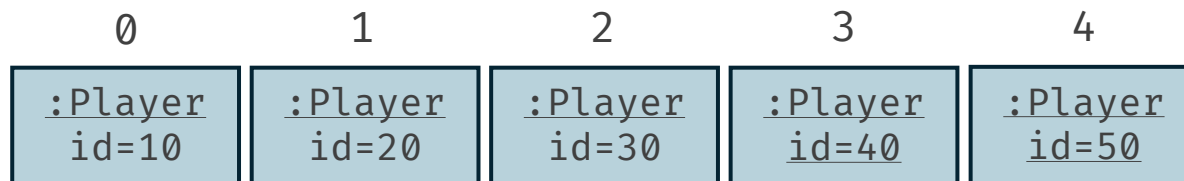


V novom poli sú move-inicializované **nové** inštancie. Ak move konštruktor nie je `noexcept` kvalifikovaný, použije sa **kopírovací konštruktor**.

```
int main() {  
    std::vector<Player> xs;  
    Player p1(10);  
    Player p2(30);  
    xs.push_back(p1);  
    xs.push_back(Player(20));  
    xs.push_back(std::move(p2));  
    xs.emplace_back(40);  
    xs.emplace_back(50);  
}
```

Vektor – expanzia

- Expanzia vektora je pomerne drahá operácia*
- Ak je to možné, chceme sa jej vyhnúť
- Metódou `reserve` môžeme predalokovať kapacitu pre dopredu známy počet prvkov



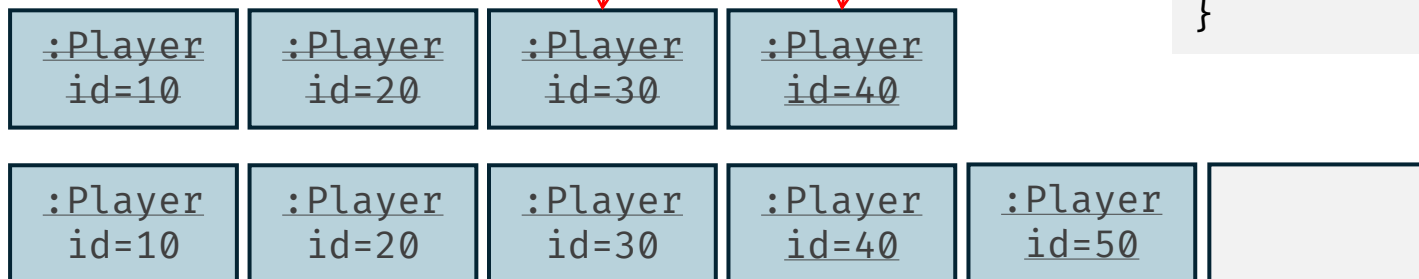
```
int main() {  
    std::vector<Player> xs;  
    Player p1(10);  
    Player p2(30);  
    xs.reserve(5);  
    xs.push_back(p1);  
    xs.push_back(Player(20));  
    xs.push_back(std::move(p2));  
    xs.emplace_back(40);  
    xs.emplace_back(50);  
}
```

Pre triviálne typy dokáže kompilátor kopírovať pamäť

Vektor – expanzia

- Dôležitou vlastnosťou vektora je, že pri expanzii zanikajú staré objekty a vznikajú nové – **mení sa ich identita**
- Táto vlastnosť môže viesť k neočakávaným chybám

```
int main() {  
    std::vector<Player> xs;  
    Player p1(10);  
    Player p2(30);  
    xs.push_back(p1);  
    xs.push_back(Player(20));  
    xs.push_back(std::move(p2));  
    xs.emplace_back(40);  
    Player *ptr1 = &xs[2];  
    Player *ptr2 = &xs[3];  
    xs.emplace_back(50); // expanzia  
    std::cout << ptr1->getId() << "\n"; // chyba  
}
```



Vektor – expanzia

- Pri práci s vektorom (či už obsahuje objektové alebo fundamentálne typy) je preto *nebezpečné* spoliehať sa na ich identitu (adresu)
- V tejto situácii (a prakticky v celej štandardnej knižnici) je vhodné nad objektami uvažovať v **hodnotovej sémantike** – teda pracovať s hodnotou/stavom a nie s identitou objektu
- Ak potrebujeme zabezpečiť zachovanie identity môžeme:
 - Vo vektore ukladať ukazovatele na objekty
 - Dopredu pred-alokovať potrebnú kapacitu, naplniť vektor a neskôr ho používať iba s `const` kvalifikátorom (aby sme zabránili expanzii)

Vektor – expanzia

- Kontajnery (a vlastne takmer všetky typy) môžeme z funkcií vracať hodnotou
- Kompilátor zabezpečí, že nevznikne zbytočný dočasný objekt, ktorý sa neskôr skopíruje do lokálnej premennej v mieste volania
- Návratová hodnota vzniká priamo v mieste volania

```
std::vector<Player> createPlayers(std::size_t n) {  
    std::vector<Player> xs;  
    xs.reserve(n);  
    for (int i = 0; i < static_cast<int>(n); ++i) {  
        xs.emplace_back(i * 10);  
    }  
    return xs;  
}  
  
int main() {  
    const std::vector<Player> xs = createPlayers(5);  
    const Player *p1 = &xs[0];  
    const Player *p2 = &xs[1];  
    std::cout << p1->getId() << "\n";  
}
```

Vektor – prístup k prvkom

- Na sprístupnenie prvkov môžeme použiť preťažený operator[] alebo metódu `at`
- `operator[]` nerobí kontrolu platnosti indexu, metóda `at` ju robí
- Metóda `front` a metóda `back` poskytujú prístup k prvému a poslednému prvku
- Metóda `data` poskytuje ukazovateľ na prvý prvok – nízkoúrovňový prístup
- Na sprístupnenie všetkých prvkov môžeme použiť cyklus `for-each`

```
int main() {  
    std::vector<int> xs;  
    xs.push_back(10);  
    xs.push_back(20);  
    xs.push_back(30);  
    std::cout << xs[0] << "\n"; // ok  
    std::cout << xs.at(1) << "\n"; // ok  
    // chyba, pravdepodobne nespadne  
    std::cout << xs[3] << "\n"; \  
    // chyba, vyhodí sa výnimka  
    std::cout << xs.at(3) << "\n";  
}
```

```
int main() {  
    std::vector<int> xs{10,20,30};  
    for (const int x : xs) {  
        std::cout << x << "\n";  
    }  
}
```

Vektor môžeme inicializovať aj pomocou syntaxe agregovanej inicializácie, keďže jeden z jeho konštruktorov preberá initializer_list.

Iterátor

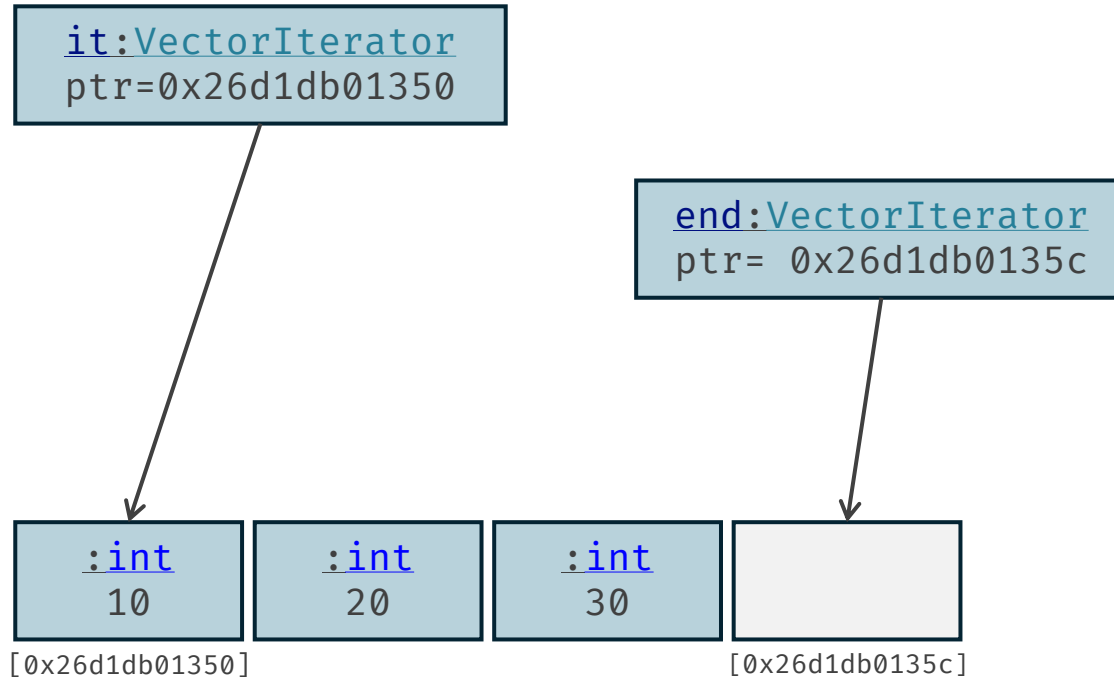
- Cyklus `for-each` funguje vďaka iterátorom – všetky kontajnery zo štandardnej knižnice poskytujú iterátory
- **Iterátor** je (proxy) objekt, ktorý **efektívne sprístupňuje** prvky kontajnera bez toho, aby odhalil **implementačné detaily** kontajnera
- Každý kontajner má svoj typ iterátora, ktorého **typ** je dostupný ako **člen triedy**
- V jazyku C++ používame (resp. cyklus `for-each` používa za nás) na prechod kontajnerom dva iterátory známe ako `begin` a `end` iterátor
- Každý kontajner poskytuje ovnomenné metódy, ktoré vracajú daný iterátor
- Iterátory sú v štandardnej knižnici veľmi dôležité – viac na nasledujúcej prednáške

Vektor – iterátor

- Prvok, na ktorý iterátor ukazuje sprístupníme cez preťažený **operator***
- Iterátor posunieme na ďalší prvok cez preťažený **operator++**
- Pre zistenie, či sa už operátor dostal na koniec ho porovnáme cez preťažený **operator!=** s **end** iterátorom

```
int main() {  
    std::vector<int> xs;  
    xs.push_back(10);  
    xs.push_back(20);  
    xs.push_back(30);  
    {  
        std::vector<int>::iterator it = xs.begin();  
        std::vector<int>::iterator end = xs.end();  
        while (it != end) {  
            const int x = *it;  
            std::cout << x << "\n";  
            ++it;  
        }  
    }  
}
```

Vektor – iterátor



End iterátor ukazuje na „prvý neplatný prvok.“

```
int main() {  
    std::vector<int> xs;  
    xs.push_back(10);  
    xs.push_back(20);  
    xs.push_back(30);  
    {  
        std::vector<int>::iterator it = xs.begin();  
        std::vector<int>::iterator end = xs.end();  
        while (it != end) {  
            const int x = *it;  
            std::cout << x << "\n";  
            ++it;  
        }  
    }  
}
```

Implementácie iterátorov pre rôzne kontajnery sa môžu a zvyčajne aj dramaticky líšia. Význam preťažených operátorov je však rovnaký.

Mapa

- Vektor pokrýva veľké množstvo prípadov použitia
- Veľkú časť zvyšných prípadov pokrýva použitie asociatívneho kontajnera (mapy, tabuľky, slovníka, ...)
- Jeho úlohou je uchovávať dvojice typu (kľúč, hodnota) a **efektívne sprístupňovať hodnoty na základe kľúča**
- Mapa preto preberá dva typové parametre **K** a **T** reprezentujúce typ kľúča a typ hodnoty
- Pre typ kľúča platia v závislosti na implementácii mapy (tabuľky) obmedzenia – musia byť porovnateľné relačnými operátormi

Mapa – implementácie

- Štandardná knižnica obsahuje dve implementácie mapy
 - usporiadaná – `std::map`
 - neusporiadaná – `std::unordered_map`*
- Obe poskytujú rovnaké rozhranie
- Obe uchovávajú inštancie `std::pair<K,T>`
- Obe mapy sú tzv. *node-based* kontajnery – údaje uchovávajú vo vrcholov – pri modifikácii zachovávajú identitu objektov
- Líšia sa vnútornou implementáciou

```
int main() {
    std::map<std::string, Player> ps;
    ps.try_emplace("Larry", 10);
    ps.try_emplace("Jack", 20);
    ps.try_emplace("Tealc", 30);
    std::cout << ps.at("Larry").getId() << "\n";
    // chyba, vyhodí sa výnimka
    std::cout << ps.at("Hammond").getId() << "\n";
    // bezpečný prístup cez iterátor
    std::map<std::string, Player>::iterator it
        = ps.find("Jack");
    if (it != ps.end()) {
        const std::pair<std::string, Player> &p = *it;
        std::cout << "key = " << p.first << "\n";
        std::cout << "value.id = "
            << p.second.getId() << "\n";
    }
}
```

Kontajner `HashMap`, ktorý poznáte z jazyka Java. *

Usporiadaná mapa

- Pri prechode usporiadanou mapou cyklom **for-each** navštívime dvojice v poradí usporiadania ich kľúčov
- Pri prechode mapou môžeme použiť syntax structured-binding na priame sprístupnenie prvkov páru uchovávaného v mape
- Túto syntax môžeme použiť aj s inými agregovanými typmi

```
int main() {  
    std::map<std::string, Player> ps;  
    ps.try_emplace("Larry", 10);  
    ps.try_emplace("Jack", 20);  
    ps.try_emplace("Tealc", 30);  
    for (const auto &[key, val] : ps) {  
        std::cout << "key = " << key << "\n";  
        std::cout << "value.id"  
            << val.getId() << "\n";  
    }  
    // Jack, Larry, Tealc  
}
```

Mapa – operátor []

- Programovacie jazyky často používajú na sprístupňovanie prvkov v mape podľa kľúča syntax **operátora[]**
- V jazyku C++ však môže byť jeho použitie zradné

```
capitals = {  
    "Germany": "Berlin",  
    "Italy": "Rome",  
    "England": "London"  
}  
print(capitals["Italy"])
```

Príklad v jazyku Python

```
public static void Main() {  
    var capitals = new Dictionary<int, string>();  
    capitals.Add("Germaý", "Berlin");  
    capitals.Add("Italy", "Rome");  
    capitals.Add("England", "London");  
    Console.WriteLine("The capital of England is: "  
        + capitals["England"]);  
}
```

Príklad v jazyku C#

Mapa – operátor []

- Programovacie jazyky často používajú na sprístupňovanie prvkov v mape podľa kľúča syntax **operátora[]**
- V jazyku C++ však môže byť jeho použitie zradné
- Ak **operator[]** nenájde hodnotu asociovanú s daným kľúčom, vytvorí ju a defaultne inicializuje – toto **môže** ale aj **nemusí byť** želané správanie

```
int main() {  
    std::map<std::string, int> counters;  
    counters.try_emplace("Gimli", 2);  
    counters.try_emplace("Legolas", 17);  
    std::cout << "Gimli got " << counters["Gimli"] << "\n";  
    std::cout << "Legolas got " << counters["Legolas"] << "\n";  
    std::cout << "Faramir got " << counters["Faramir"] << "\n";  
}  
  
// Gimli got 2  
// Legolas got 17  
// Faramir got 0
```

Čas a dátum



Čas a dátum

- Práca s časom a dátumom je v informatike komplikovaná aj kvôli existencii:
 - Časových pásiem
 - Priestupných rokov
 - Priestupných sekúnd
- O úskaliach práce s časom si môžete viac pozrieť v nasledujúcich prednáškach:
 - Martin Mareš – Programátoři versus realita
 - Tom Scott – The Problem with Time & Timezones
 - Cassio Neri – Implementing Fast Calendard Algorithms

Čas a dátum

- Čas ukladáme ako počet neprestupných sekúnd od **00:00:00 1.1.1970** (Unix Epoch)
- Táto hodnota je známa aj ako Unix Timestamp
- S jeho uchovávaním je spojených niekoľko problémov
- Ak sa na uchovanie použije 32-bitový celočíselný typ so znamienkom, posledný čas, ktorý je možné reprezentovať bude **03:14:08 19.1.2038** – táto situácia je známa ako Year 2038 problem

Čas a dátum v jazyku C++

- V jazyku C++ používame na prácu s časom a dátumom (od C++11) knižnicu `<chrono>`
- Knížnica poskytuje niekoľko typov hodín
 - system_clock – poskytujú aktuálny systémový čas
 - steady_clock – poskytujú stabilný čas (napríklad od posledného reštartu systému)
 - high_resolution_clock – poskytujú stabilný čas s najvyšším možným rozlíšením
- Všetky hodiny poskytujú statickú členskú funkciu `now`, ktorá vráti ich aktuálny čas ako inštanciu triedy time_point
- `time_point` je možné použiť na zobrazenie času alebo na ďalšie výpočty

Čas a dátum v jazyku C++

- Počiatočná skúsenosť s knižnicou je pomerne náročná, pretože vo veľkom využíva šablóny
- Benefitom je typová bezpečnosť – nedochádza k neželaným konverziám jednotiek

```
int main() {  
    namespace ch = std::chrono;  
    ch::system_clock::time_point now = ch::system_clock::now();  
    ch::time_point<ch::system_clock, ch::days> today = ch::floor<ch::days>(now);  
    ch::year_month_day ymd{today};  
    ch::hh_mm_ss hhmss{now - today};  
    std::cout << std::format("{:%Y-%m-%d}", ymd) << "\n"  
                << std::format("{:%H:%M:%S}", hhmss) << '\n';  
}
```

Stopky v jazyku C++

- Knížnica chrono je vhodná aj na meranie času medzi dvoma okamihmi
- Trvanie je reprezentované triedou `std::chrono::duration`

```
int main() {  
    namespace ch = std::chrono;  
    ch::high_resolution_clock::time_point before = ch::high_resolution_clock::now();  
    longRunningFunction();  
    ch::high_resolution_clock::time_point after = ch::high_resolution_clock::now();  
    ch::nanoseconds elapsed = ch::duration_cast<ch::nanoseconds>(after - before);  
    std::cout << "Elapsed time " << elapsed.count() << "ns\n";  
    std::cout << std::format("Elapsed time {:%Q%q}\n", elapsed) << "\n";  
}
```

Súborový systém



Súborový systém

- Spôsob práce so súborovým systémom je závislý na systéme, pre ktorý programujeme (Windows API, POSIX, ...)
- (od C++17) Knižnica `<filesystem>` abstrahuje od rozhrania systému a ponúka nástroje na prácu so súborovým systémom
- Najdôležitejšou triedou je trieda `std::filesystem::path`, pomocou ktorej interagujeme so súbormi a priečkami

```
int main() {  
    namespace fs = std::filesystem;  
    fs::path dir("2025/results");  
    fs::create_directories(dir);  
    fs::path data("data.txt");  
    fs::path dest = dir / data;  
    std::ofstream ofst(dest);  
    ofst << "my results\n";  
}
```

Súborový systém

- Rozhranie triedy `path` umožňuje zistiť príponu súboru, názov, absolútnu cestu, ...
- Pomocou voľných funkcií môžeme zisťovať vlastnosti súborov, mazať ich, ...
- Pomocou iterátora priečinkov môžeme prechádzať všetky súbory v danom umiestnení
- Všetko bez ohľadu na platformu

```
int main() {  
    namespace fs = std::filesystem;  
    fs::path dir(".");  
    if (!fs::exists(dir)) {  
        return 1;  
    }  
    for (const auto &entry :  
         fs::recursive_directory_iterator(dir))  
    {  
        fs::path p = entry.path();  
        if (p.extension() == ".tmp") {  
            std::cout << "Deleting " << p << "\n";  
            fs::remove(p);  
        }  
    }  
}
```


Náhodné čísla

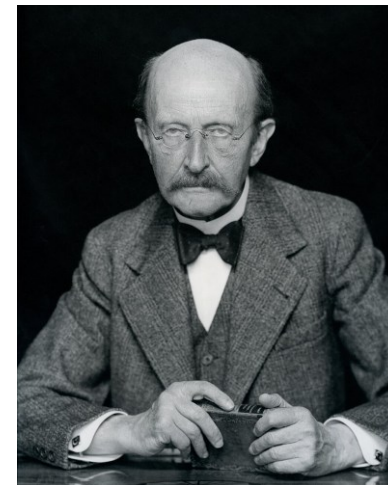
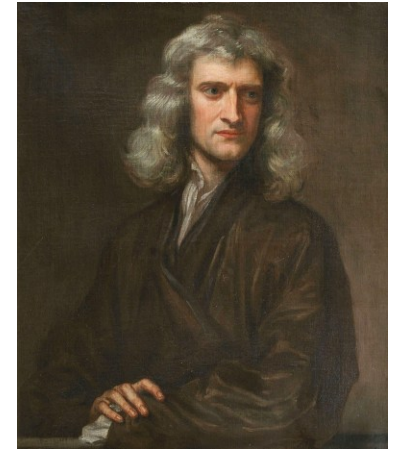


Náhodné čísla

- Generovanie náhodných čísiel je dôležitou súčasťou informatiky
- Veľmi dôležité je v oblastiach
 - Počítačových simulácií
 - Hier
 - Kryptografii
- Väčšina programovacích jazykov vo svojej štandardnej knižnici poskytuje aspoň základnú podporu pre generovanie (pseudo)náhodných čísel
- Štandardná knižnica jazyka C++ je v tomto nadpriemerná

Náhodné čísla?

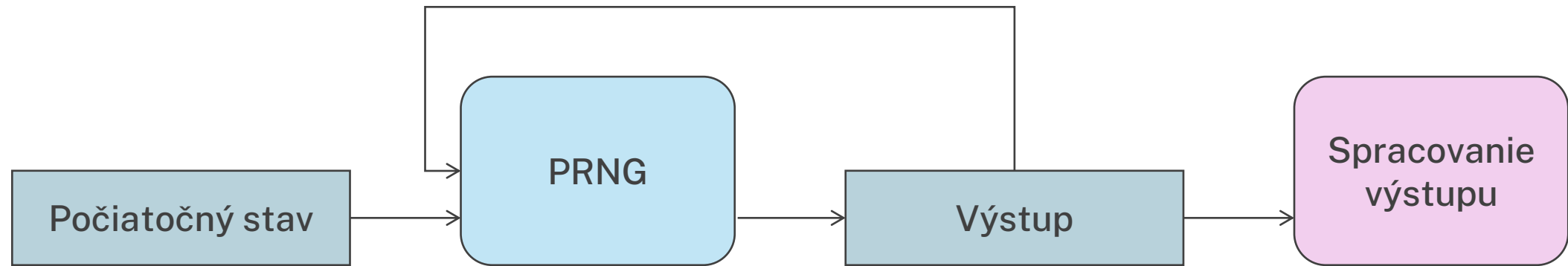
- Zaujímavou otázkou je, či je vôbec možné generovať skutočne náhodné čísla
- Odpoveď musíme hľadať vo fyzike
- Podľa toho, v ktorom roku a koho by sme sa otázku by sme dostali rôzne odpovede



Generovanie (psuedo)náhodných čísel

- Na generovanie skutočne náhodných čísel by sme museli použiť hardvérové zariadenie (alebo jeho sprostredkovanie)
- Preto sa ďalej zameriame iba na generovanie pseudonáhodných čísel
- Pseudonáhodné čísla spĺňajú určité **štatistické vlastnosti**, vďaka čomu ich môžeme použiť napríklad v simuláciách a v kryptografii
- Všetky vygenerované čísla sú však dopredu určené počiatočným stavom generátora – preto predpona **pseudonáhodné**

Princíp fungovania



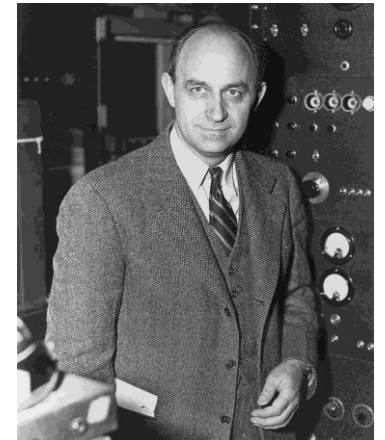
PRNG z angl. Pseudo Random Number Generator.

Počiatočný stav nazývame aj seed.
Určuje všetky vygenerované čísla.
Môže pochádzať z rôznych zdrojov.

Výstup má formu bajtov (čísla).
Výstup je zároveň vstupom pre generovanie ďalšieho čísla.

Generovanie (psuedo)náhodných čísel

- *Kvalita* vygenerovaných pseudonáhodných čísel závisí od implementácie PRNG
- Prvotné implementácie a ich výskum pochádzajú z 20. storočia z obdobia 2. sv. vojny
- Pre ich obrovské uplatnenie sa implementácie postupne zdokonaľovali



Lineárny Kongruenčný generátor

- Lineárny kongruenčný generátor patrí medzi základné implementácie PRNG
- Veľmi dobre ilustruje princíp fungovania
- Jeho kvalita závisí od zvolených konštánt
- Kvalitu môžeme určovať aj podľa toho, ako rýchlo sa generátor dostane do cyklu alebo skonverguje

$$Z_{n+1} = (1664525 * Z_n + 1013904223) \bmod 2^{32}$$

$Z_0(\text{seed})$	144
Z_1	1253595823
Z_2	1459898434
Z_3	1793256121
Z_4	4287337668
Z_5	1525651795
Z_6	1254870678

Pseudonáhodné čísla v jazyku C++

- V jazyku C++ poskytuje nástroje na prácu s pseudonáhodnými číslami (od C++11) knižnica `<random>`
- Knížnica správne rozlišuje PRNG a rozdelenie pravdepodobnosti

```
int main() {
    std::default_random_engine diceRng(std::random_device{}());
    std::default_random_engine tempRng(std::random_device{}());
    std::uniform_int_distribution<int> diceDist(1,6);
    std::normal_distribution<double> tempDist(20.0, 5.0);
    for (int i; i < 10; ++i) {
        std::cout << std::format("Dice throw {}\n", diceDist(diceRng));
        std::cout << std::format("Under temperature {:.2f} °C\n", tempDist(tempRng));
    }
}
```


Pseudonáhodné čísla v jazyku C++

- Knižnica poskytuje rôzne implementácie PRNG – v príklade využívame alias `default_random_engine`, ktorý zastupuje jeden z nich
- Pre získanie náhodného čísla musíme generátor spojiť s reprezentáciou požadovaného rozdelenia pravdepodobnosti

```
int main() {
    std::default_random_engine diceRng(std::random_device{}());
    std::default_random_engine tempRng(std::random_device{}());
    std::uniform_int_distribution<int> diceDist(1,6);
    std::normal_distribution<double> tempDist(20.0, 5.0);
    for (int i; i < 10; ++i) {
        std::cout << std::format("Dice throw {}\n", diceDist(diceRng));
        std::cout << std::format("Under temperature {:.2f} °C\n", tempDist(tempRng));
    }
}
```

Zásady práce s PRNG

- Každý modelovaný náhodný jav by mal byť reprezentovaný **samostatným PRNG**
 - Ak, napríklad, simulujeme hody tromi kockami, každá kocka by mala mať svoj PRNG
- Je potrebné vyhýbať sa opakovanej inicializácii a zrušení PRNG
- PRNG by sme mali inicializovať raz **kvalitným počiatočným stavom** a potom používať **rovnakú inštanciu**
 - Napríklad nie je vhodné, aby bol PRNG iba lokálnou premennou, ktorá vzniká a zaniká pri každom volaní funkcie
- Pri nedodržaní pravidiel môžeme dostávať výstup, ktorý nebude mať správne štatistické vlastnosti – to môže ovplyvniť, napríklad, kvalitu výsledkov simulácie

Zhrnutie

- Použitie statickej alebo dynamickej linkovanej knižnice ovplyvňuje vlastnosti spustiteľného súboru
- Je dôležité vedieť, aké závislosti náš projekt má a akým spôsobom sú k nemu pripájané
- Štandardná knižnica poskytuje nástroje pokrývajúce väčšinu bežných prípadov použitia
- Je dobré mať základný prehľad o tom, čo všetko obsahuje
- Obzvlášť pri práci s údajmi, pri ktorých je veľký priestor na chybu (čas a dátum), je vhodné použiť overené knižničné riešenie

Ďakujem za pozornosť

Priestor na otázky

